

Compte rendu de TP

Firestore — Authentication

Étudiant Délio Duchesne-Brajon — Groupe 1
Bloc Bloc 2 — Développement
Outils Flutter · Cloud Firestore · Firebase Auth
Date 25 septembre 2026

SOMMAIRE

1. Objectif du TP	2
2. Cloud Firestore — structuration de la base	2
2.1. Création de la première collection	2
2.2. Collection items	3
3. Application Flutter — Firebase Todolist	3
3.1. Écriture des données depuis l'application	3
3.2. Lecture et affichage de la collection	4
3.3. Structure du code.....	5
3.4. Todolist fonctionnelle.....	6
4. Firebase Authentication	6
4.1. Page de connexion	6
4.2. Inscription d'un second compte	7
4.3. Page de profil	8
5. Profil utilisateur stocké dans Firestore.....	8
6. Conclusion	9

1. Objectif du TP

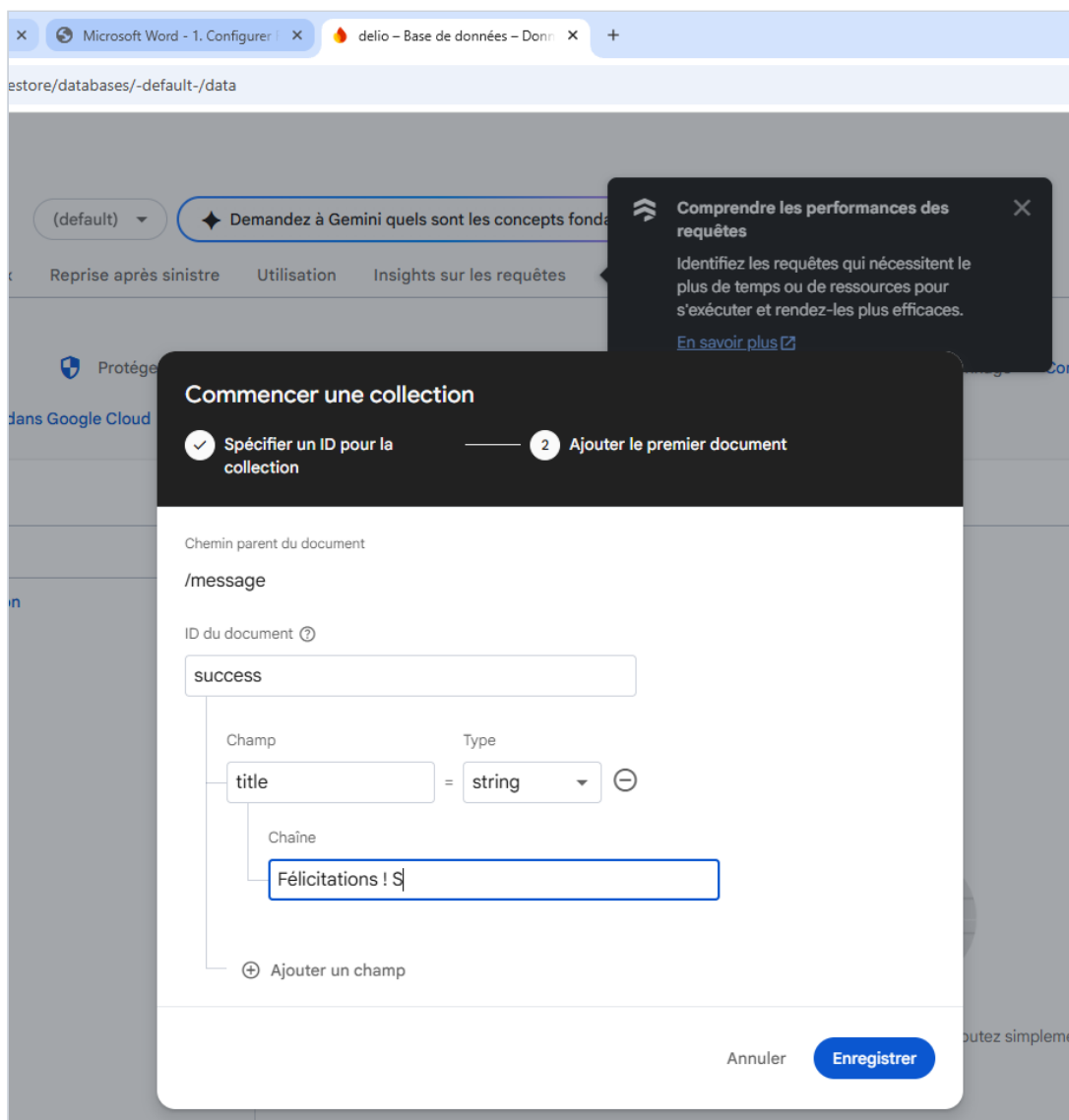
Découvrir Firebase comme back-end d'une application Flutter : structurer une base Cloud Firestore, y lire et y écrire depuis l'application, puis gérer les comptes utilisateurs avec Firebase Authentication.

L'application développée est une liste de tâches, « Firebase Todolist », exécutée dans Chrome via flutter run.

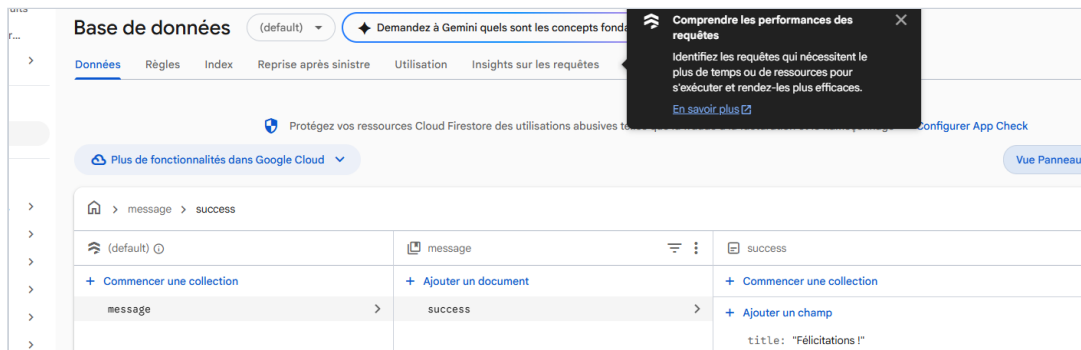
2. Cloud Firestore — structuration de la base

2.1. Création de la première collection

La console Firebase permet de créer une collection puis d'y ajouter un premier document. La collection message reçoit le document success, contenant un champ title de type *string* avec la valeur « Félicitations ! ».

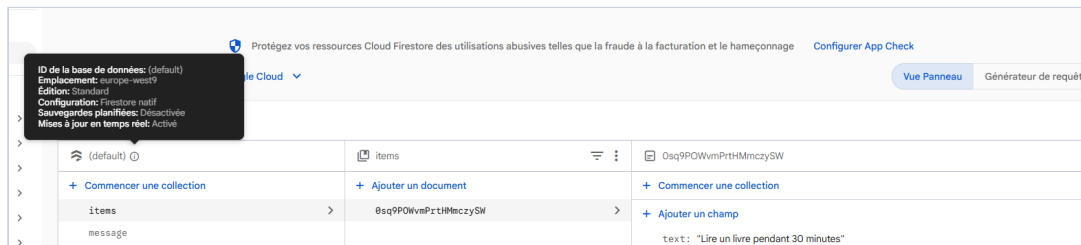


Le document apparaît ensuite dans la vue en trois colonnes de Firestore : la collection, le document, puis ses champs.



2.2. Collection items

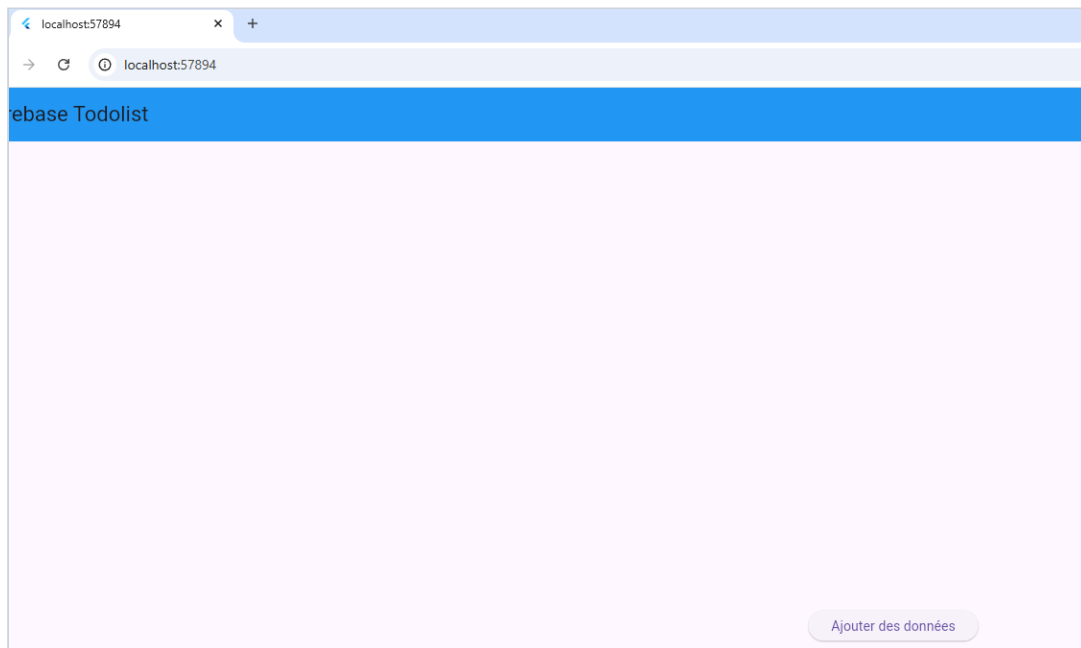
Une seconde collection, `items`, accueille les tâches de la future liste. Chaque document possède un identifiant généré automatiquement et un champ `text`.



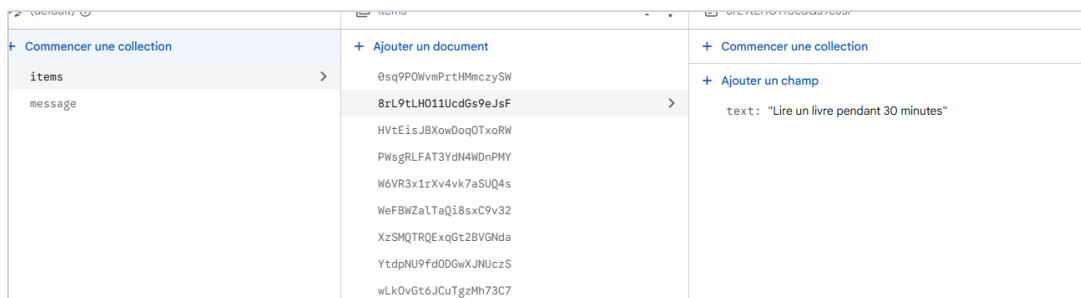
3. Application Flutter — Firebase Todolist

3.1. Écriture des données depuis l'application

L'application est lancée dans le navigateur. Elle se compose d'une barre de titre et d'un bouton « Ajouter des données » qui écrit un document dans la collection `items`. La liste est encore vide à ce stade.

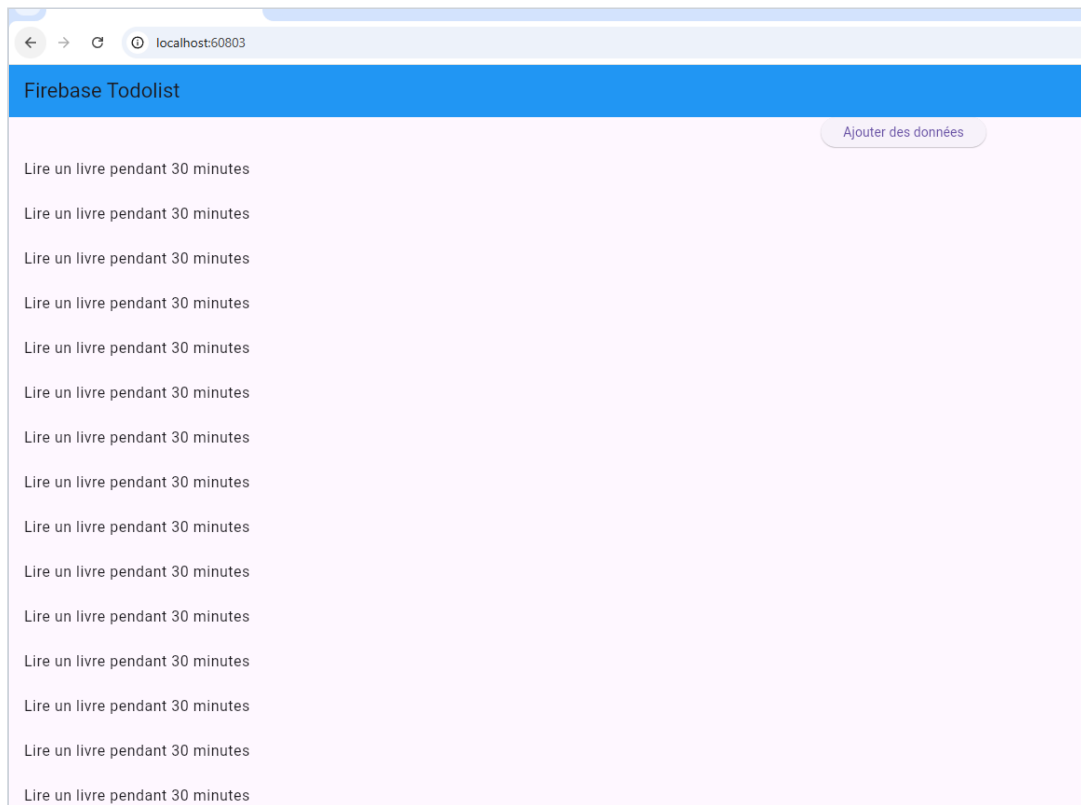


Chaque appui sur le bouton crée un nouveau document, ce que confirme la console : la collection se remplit d'entrées identiques.



3.2. Lecture et affichage de la collection

L'application lit ensuite la collection et affiche chaque document sous forme de ligne. Les enregistrements créés précédemment apparaissent bien dans l'interface, ce qui valide la liaison entre Flutter et Firestore dans les deux sens.



3.3. Structure du code

Le point d'entrée déclare la référence à la base avec `Firestore.instance`, puis construit l'interface : un `Scaffold` dont l'`AppBar` porte le titre de l'application, un `ElevatedButton` appelant `addDataToFirebase()`, et un `Expanded` contenant le widget `ListSection()` chargé de l'affichage.

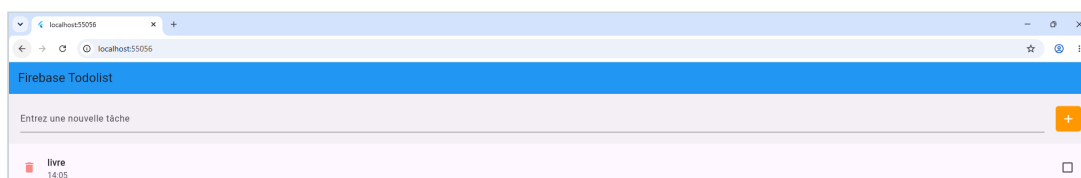
```

74 find databaseReference = FirebaseFirestore.instance;
75
76 @override
77 Widget build(BuildContext context) {
78   return MaterialApp(
79     debugShowCheckedModeBanner: false,
80     theme: ThemeData(primarySwatch: Colors.amber),
81     home: Scaffold(
82       appBar: AppBar(
83         title: const Text('Firebase Todolist'),
84         backgroundColor: Colors.blue,
85       ), // AppBar
86       body: Column(
87         children: [
88           Center(
89             child: ElevatedButton(
90               onPressed: () {
91                 addDataToFirebase();
92               },
93               child: const Text('Ajouter des données'),
94             ), // ElevatedButton
95           ), // Center
96           Expanded(
97             child: ListSection(),
98           ), // Expanded
99         ],
100       ), // Column
101     ), // Scaffold
102   ); // MaterialApp
103 }
104

```

3.4. Todolist fonctionnelle

Le bouton de test est remplacé par un champ de saisie permettant d'entrer une nouvelle tâche, accompagné d'une case à cocher sur chaque ligne. L'application devient une véritable liste de tâches.



4. Firebase Authentication

4.1. Page de connexion

Une page de connexion est ajoutée à l'application. Un premier compte est créé dans la console Firebase, visible dans l'onglet Authentication avec son adresse, sa date de création et son identifiant utilisateur.

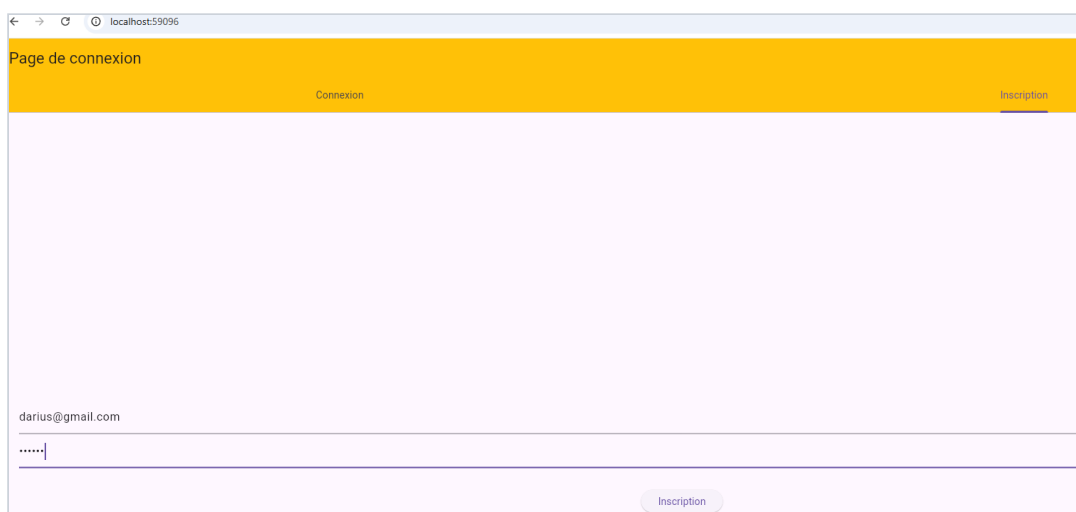


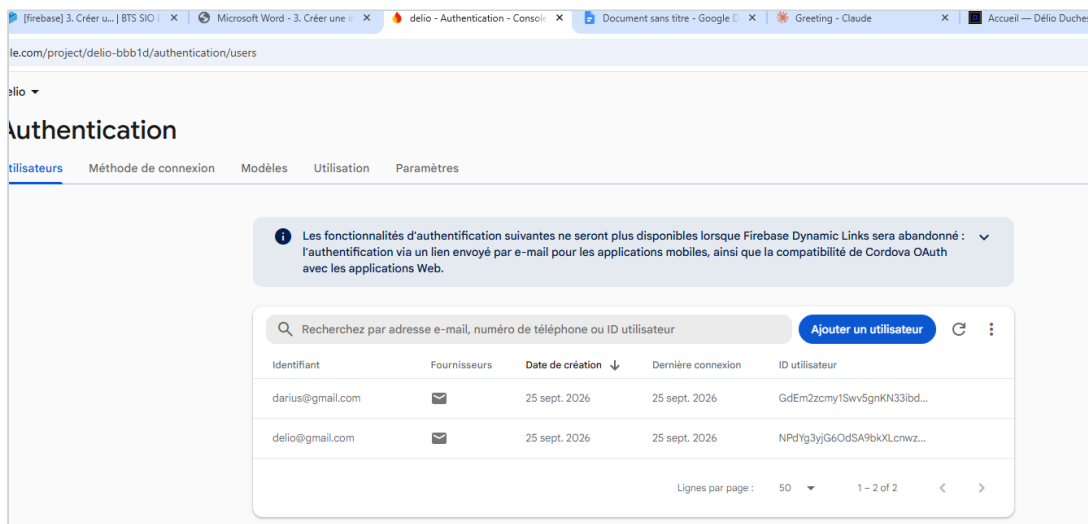
La saisie de l'adresse et du mot de passe dans l'application permet ensuite de s'authentifier.



4.2. Inscription d'un second compte

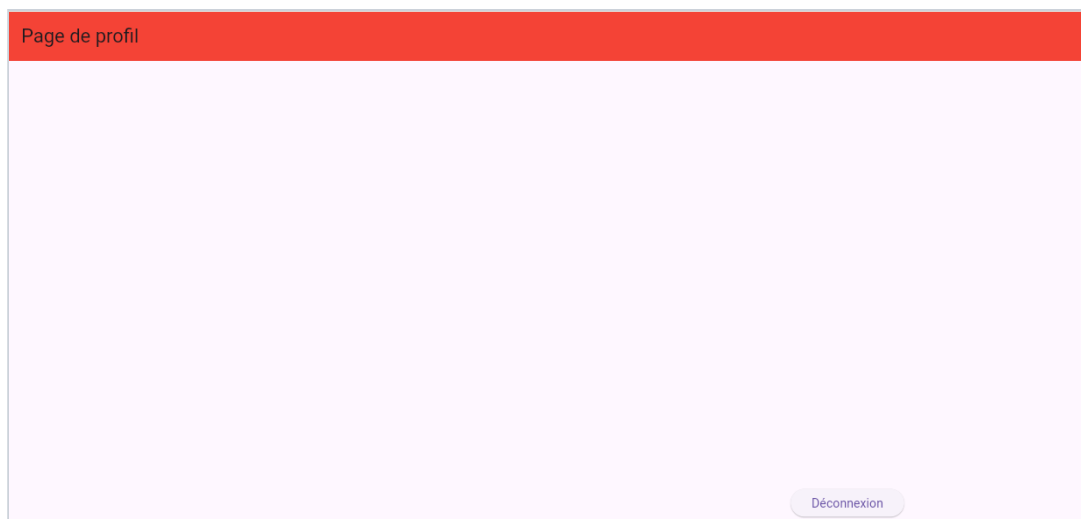
La page propose également un onglet d'inscription. La création du compte `darius@gmail.com` depuis l'application se répercute immédiatement dans la console, qui compte alors deux utilisateurs.





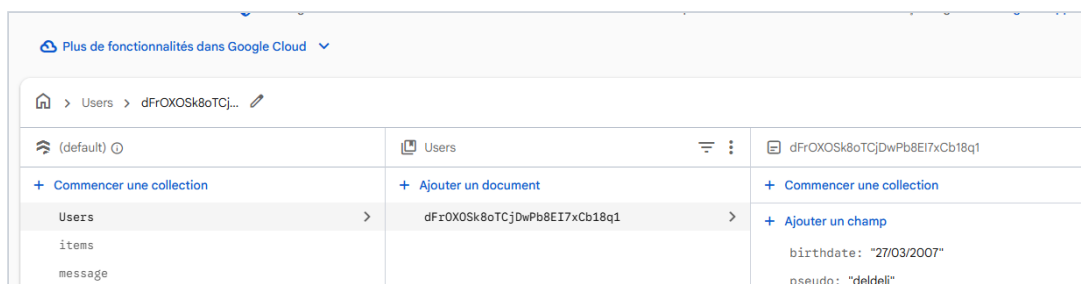
4.3. Page de profil

Une fois connecté, l'utilisateur accède à une page de profil munie d'un bouton de déconnexion.

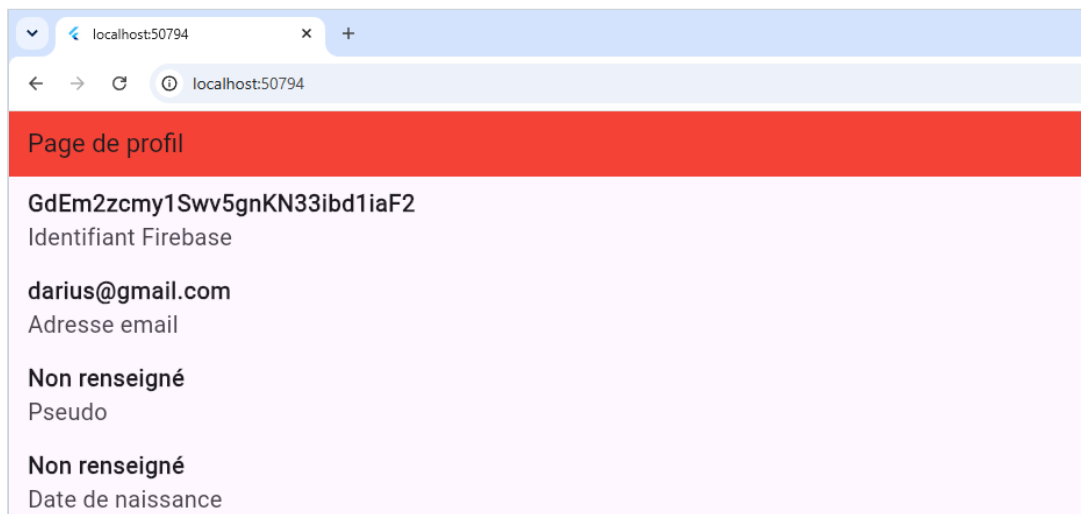


5. Profil utilisateur stocké dans Firestore

Une troisième collection, users, est destinée aux informations complémentaires du compte : un pseudo et une date de naissance, que l'authentification Firebase ne gère pas.



La page de profil affiche l'identifiant Firebase et l'adresse e-mail, correctement récupérés depuis le compte authentifié, mais indique « Non renseigné » pour le pseudo et la date de naissance.



L'explication apparaît en comparant les identifiants : le document de la collection users porte un identifiant généré automatiquement, alors que la page de profil recherche un document dont l'identifiant correspond à l'UID du compte connecté. Aucune correspondance n'est donc trouvée. Pour que le profil se remplisse, le document doit être créé avec l'UID de l'utilisateur comme identifiant, et non avec un identifiant automatique.

6. Conclusion

Ce TP couvre l'essentiel de Firebase côté application : structuration d'une base Firestore en collections et documents, lecture et écriture depuis Flutter, puis gestion des comptes avec Authentication.

Le point le plus instructif concerne la liaison entre les deux services : Authentication et Firestore sont indépendants, et c'est à l'application de faire le lien en utilisant l'UID du compte comme identifiant de document. C'est précisément ce qui manque au profil de ce TP, et le premier point à corriger par la suite.